🛡️ MIRAGE Junior — Master Project Documentation & Architectural Guide

Comprehensive Technical Deep-Dive: Code, Algorithms, Page Explanations & Defense Rationale

> **Project Name:** MIRAGE Junior — Intelligent Deception Network with Threat Scoring & Auto-Quarantine
> **Authors:** Jaskaran Bandral (Team Leader, Backend & Core Engine) & Relish Sharma (Teammate, Dashboard & Integration)
> **Academic Context:** 3rd Year B.Tech / B.E. Cyber Security & Computer Science
> **Live Showcase:** mirage-junior.vercel.app
> **Version:** 2.1 Production Verified

📑 Table of Contents

1. Executive Summary & Core Philosophy
2. Complete Dashboard Page-by-Page Deep Dive
3. 2.1 Overview Command Center (`/`)
4. 2.2 Full Attack Telemetry Log (`/attacks`)
5. 2.3 Attacker Profile & Timeline Replay (`/attacker/<ip>`)
6. 2.4 Quarantine Management (`/quarantine`)
7. 2.5 Honeytoken Tripwire Hub (`/honeytokens`)
8. 2.6 System Settings & Control Panel (`/settings`)
9. Backend Engine & Code Modules Explained
10. 3.1 Behavioral Threat Scoring Engine (`engine/scoring_engine.py`)
11. 3.2 Automated Quarantine & Firewall Engine (`engine/quarantine.py`)
12. 3.3 Multi-Protocol Honeypot Traps (`honeypots/`)
13. 3.4 Honeytoken Watchdog Monitor (`engine/honeytoken_monitor.py`)
14. 3.5 GeoIP Intelligence Enricher (`engine/geo_enricher.py`)
15. 3.6 MITRE ATT&CK Mapping Engine (`engine/mitre_tagger.py`)
16. 3.7 Real-Time Alert Manager (`engine/alert_manager.py`)
17. 3.8 Concurrency-Safe Database Layer (`database/`)
18. College Viva & Evaluator Defense Guide

---

1. Executive Summary & Core Philosophy

The Fundamental Problem with Traditional Honeypots:

Most academic honeypot projects are **purely passive**. They bind a socket, capture credentials, and write them into a log file or database. When an attack happens, traditional honeypots do nothing to defend the network.

The MIRAGE Junior Paradigm Shift:

MIRAGE Junior transforms honeypots from a **passive listener** into an **Active Cyber Deception & Autonomous Response System**: 1. **Behavioral Risk Quantification:** Every attacker action accumulates dynamic risk points (0–100 scale). 2. **Autonomous Response:** The moment an attacker crosses the threshold (**≥ 25 points**), the system automatically adds an OS firewall block rule (`netsh advfirewall` on Windows / `iptables` on Linux). 3. **High-Fidelity Alarms (Zero False Positives):** Through planted Honeytokens (`.env` , `credentials.xlsx` , `backup_database.sql`), any touch is guaranteed malicious intent. 4. **Instant Human-in-the-Loop Notification:** Real-time push alerts delivered to the SOC analyst's smartphone via Telegram Bot API.

---

2. Complete Dashboard Page-by-Page Deep Dive

For each page, we analyze: * **Kyu Kiya? (Objective)** * **Kisliye Kiya? (Purpose & Benefits)** * **Kaise Kiya? (Implementation & Code)** * **Reason Kya Tha? (Design Rationale)**

2.1 Overview Command Center (`/`)

🎯 Kyu Kiya? (Objective)

Security operators in a SOC need an executive, real-time command center where they can assess network security posture at a single glance without sifting through thousands of log rows.

💡 Kisliye Kiya? (Purpose & Benefits)

- Instant visibility into high-level metrics: Total Hits, Unique Attacker IPs, Stolen Credentials Captured, Average Threat Score, and Currently Quarantined IPs.
- Visual charts for Threat Distribution (Severity breakdown) and Top Attacking Countries.
- Live attack feed that automatically updates via asynchronous polling every 5 seconds.

🛠️ Kaise Kiya? (Implementation & Code)

- **Backend Route:** `dashboard/routes.py` function `overview()` .
- **SQL Aggregations:** `python total_hits = db.execute("SELECT COUNT(*) FROM events").fetchone()[0] unique_ips = db.execute("SELECT COUNT(*) FROM attackers").fetchone()[0] creds_captured =`

`db.execute("SELECT COUNT(*) FROM credentials").fetchone()[0] avg_score = db.execute("SELECT AVG(threat_score) FROM attackers WHERE threat_score > 0").fetchone()[0] quarantined = db.execute("SELECT COUNT(*) FROM quarantine_log WHERE is_active = 1").fetchone()[0]`

- **Visualizations:** `dashboard/static/js/charts.js` uses Chart.js to render a doughnut chart for Threat Severity (`Scanner`, `Suspicious`, `Active Threat`, `Critical`) and a bar chart for Top Origin Countries.
- **Dynamic Live Feed:** AJAX fetch to `/api/stats` and `/api/events` every 5 seconds updates `#stat-total-hits`, `#live-attack-table`, and `#quarantine-table` without refreshing the page.

🧠 Reason Kya Tha? (Design Rationale)

Why use Chart.js with dark neon cyber aesthetics? College evaluators judge dashboards based on industry relevance. A dark neon terminal UI matches real-world SIEMs like Splunk, Datadog, and Microsoft Sentinel.

2.2 Full Attack Telemetry Log (`/attacks`)

🎯 Kyu Kiya? (Objective)

Security analysts must investigate historical attack logs in depth, filter by specific protocols (e.g. only SSH attacks), search for malicious IPs or payload signatures, and export raw data for compliance audits.

💡 Kisliye Kiya? (Purpose & Benefits)

- Solved the "information overload" problem by implementing **clean 10-records-per-page pagination**.
- Protocol filter chips allow instant toggling between `All`, `SSH`, `HTTP`, `FTP`, and `Honeytokens`.
- Dedicated **Export CSV** button allowing immediate compliance download of `mirage_events.csv`.

🛠️ Kaise Kiya? (Implementation & Code)

- **Backend Route:** `dashboard/routes.py` function `attacks()`.
- **Dynamic SQL Query with Offset Pagination:** ```python page = request.args.get('page', 1, type=int) per_page = 10 offset = (page - 1) * per_page protocol = request.args.get('protocol', '').strip().upper() search = request.args.get('search', '').strip()

Dynamic SQL builder filters both protocol and search terms safely via parameterized queries fetch_sql = """SELECT e.*, a.country, a.city, a.classification, a.threat_score FROM events e LEFT JOIN attackers a ON e.ip_address = a.ip_address WHERE ... ORDER BY e.timestamp DESC LIMIT ? OFFSET ?""" `` - **UI Template:** [dashboard/templates/attacks.html](file:///c:/Users/HP/Downloads/mirage-junior-main/mirage-junior-main/dashboard/templates/attacks.html) displays: - Timestamp (with clock icon) - Clickable Attacker IP (links to /attacker/) - Origin Country - Protocol badge (🌐 HTTP, 💻 SSH, 📁 FTP, ☣️ BAIT) - Action / Payload details - Dynamic Threat Points (+15 pts, +25 pts) - MITRE ATT&CK technique pill - Attacker risk score badge (CRITICAL, ACTIVE`, etc.)

🧠 Reason Kya Tha? (Design Rationale)

Why 10 per page instead of infinite scroll? Infinite scroll crashes browsers with thousands of DOM nodes. 10-per-page pagination provides fast query execution via SQLite `LIMIT ? OFFSET ?` and preserves state across back/forward navigation.

2.3 Attacker Profile & Timeline Replay (`/attacker/<ip>`)

🎯 Kyu Kiya? (Objective)

An IP address is just a number. Security analysts need a **comprehensive dossier** on a specific attacker: Where are they located? What ISP do they use? What credentials did they try? And most importantly: **What was the exact chronological sequence of their attack?**

💡 Kisliye Kiya? (Purpose & Benefits)

- Forensic investigation deep-dive.
- **Visual Attack Timeline Replay:** Demonstrates an "attack narrative" showing the escalation from initial reconnaissance to credential theft to quarantine.
- Captured Credentials Table: Shows every username/password combination tested by this specific IP.

🛠️ Kaise Kiya? (Implementation & Code)

- **Backend Route:** `dashboard/routes.py` function `attacker(ip)`.
- **Timeline Engine:** `dashboard/static/js/timeline.js` function `renderTimeline(events, container)`.
- Computes cumulative running threat score across the chronological attack stream.
- Color-codes timeline nodes based on cumulative threat score:
  - `< 21 pts`: Green dot (Scanner)
  - `< 51 pts`: Yellow dot (Suspicious)
  - `< 76 pts`: Orange dot (Active Threat)
  - `≥ 76 pts`: Red pulsing dot (Critical Threat)

🧠 Reason Kya Tha? (Design Rationale)

Evaluators love stories. Instead of telling the teacher "this IP attacked us", you click the IP and show: *"Sir, at 11:00 AM the attacker scanned port 8080. At 11:02 AM they attempted brute-force SSH. At 11:05 AM they touched the `.env` honeytoken. At that exact second, their score hit 88 and the system automatically quarantined them."*

2.4 Quarantine Management (`/quarantine`)

🎯 Kyu Kiya? (Objective)

To manage, monitor, and audit the **Active Defense Layer**. Security operators need full visibility into who is currently jailed by the firewall and the ability to manually pardon/unblock legitimate IPs.

💡 Kisliye Kiya? (Purpose & Benefits)

- Bridges the gap between honeypot detection and firewall enforcement.
- Displays Active Quarantines vs Historical Expired Quarantines.
- Features a working asynchronous **Unblock Button** (`POST /api/unblock/<ip>`).

🛠️ Kaise Kiya? (Implementation & Code)

- **Backend Route:** `dashboard/routes.py` functions `quarantine()` and `api_unblock_ip(ip)` .
- **Database Tracking:** Every quarantine event is stored in the `quarantine_log` table:
- `blocked_at` , `expires_at` , `reason` , `threat_score` , `is_active` , `unblocked_by` .
- **Firewall Integration:** Calling `quarantine_manager.unblock_ip(ip)` deletes the firewall rule (`netsh advfirewall firewall delete rule name="MIRAGE_BLOCK_{ip}"`) and updates database record `is_active = 0` .

🧠 Reason Kya Tha? (Design Rationale)

Why support auto-expiry? In the real world, hackers use dynamic residential IPs or botnets that change IPs frequently. Blocking an IP permanently creates bloated firewall tables and risks blocking innocent future users. A 1-hour default auto-expiry (`QUARANTINE_DURATION = 3600`) is industry best practice.

2.5 Honeytoken Tripwire Hub (`/honeytokens`)

🎯 Kyu Kiya? (Objective)

To monitor planted digital baits across HTTP, FTP, and local filesystems, providing an unalterable audit trail of decoy interactions.

💡 Kisliye Kiya? (Purpose & Benefits)

- Displays Planted Token Inventory (`.env` , `credentials.xlsx` , `backup_database.sql` , `admin_passwords.txt` , `id_rsa`).
- Real-time trigger counters and Trigger History table.

🛠️ Kaise Kiya? (Implementation & Code)

- **Backend Route:** `dashboard/routes.py` function `honeytokens()` .
- **Multi-Vector Triggering:**
- Web hit on `/.env` → logged via `http_decoy.py` .
- FTP download of `credentials.xlsx` → logged via `ftp_trap.py` .
- Direct local file touch on `bait/*` → logged via Python `watchdog` in `honeytoken_monitor.py` .

🧠 Reason Kya Tha? (Design Rationale)

The Attacker's Dilemma: In classical defense, the defender must protect 100% of the surface, while the attacker only needs 1 flaw. Honeytokens **invert this asymmetry** — the defender only needs to place 1 tempting bait; the moment the attacker touches it, their entire campaign is compromised.

2.6 System Settings & Control Panel (`/settings`)

🎯 Kyu Kiya? (Objective)

To allow administrators to tune the security parameters, update Telegram Bot credentials, and whitelist subnets **without editing code or restarting server daemons**.

💡 Kisliye Kiya? (Purpose & Benefits)

- Input fields for `TELEGRAM_BOT_TOKEN` and `TELEGRAM_CHAT_ID`.
- Tunable `Quarantine Score Threshold` (default: **25 points**).
- IP Whitelist textarea supporting individual IPs and CIDR ranges (`192.168.0.0/16` , `10.0.0.0/8`).
- Flash notifications confirming updates on submit.

🛠️ Kaise Kiya? (Implementation & Code)

- **Backend Route:** `dashboard/routes.py` function `settings()` with `GET` and `POST` methods.
- **Persistent Whitelist File:** Writes updated IP list directly to `config/whitelist.json` .
- **In-Memory Configuration:** Updates `config.settings.QUARANTINE_THRESHOLD` on the fly.

🧠 Reason Kya Tha? (Design Rationale)

Why is CIDR whitelisting vital? If a honeypot is deployed on an enterprise intranet, an administrator running a vulnerability scan from their workstation might trigger the honeypot. Whitelisting the admin subnet prevents accidental self-blocking.

---

3. Backend Engine & Code Modules Explained

3.1 Behavioral Threat Scoring Engine (`engine/scoring_engine.py`)

Algorithm & Mathematical Model:

The Threat Score $S(A)$ for an attacker IP A is calculated dynamically over all recorded interactions:

$$S(A) = \min\left(100, \sum_{i=1}^{n} W(E_i) + M_{\text{lateral}}\right)$$

Where: - $W(E_i)$ is the pre-configured point weight for event type E_i defined in `config/scoring_rules.json` . - $M_{\text{lateral}} = 15$ if the attacker targets multiple distinct ports/services ($\text{distinct}(P) > 1$). - The total score is strictly capped at 100.

Threat Weight Table:

Attacker Action	Event Type	Points Added	Justification
Initial Connection	`connection`	**+5**	Low noise, standard network scan
Credential Brute-Force	`login_attempt`	**+8**	Targeted authorization bypass attempt
Interactive Shell Access	`shell_access`	**+15**	Critical boundary breach
Recon Command Execution	`shell_command`	**+5**	Running discovery tools (`whoami` , `id`)
Password File Theft	`cat_passwd`	**+10**	Privilege escalation preparation
Sensitive File Download	`ftp_download`	**+15**	Exfiltration of corporate data
Honeytoken Trigger	`honeytoken_trigger`	**+25**	High-fidelity malicious breach indicator
Cross-Service Scanning	`lateral_scan`	**+15**	Lateral movement across SSH, HTTP, and FTP

Auto-Quarantine Trigger:

```
# Quarantine attacker if score meets or exceeds threshold (25 pts)
threshold = getattr(settings_cfg, 'QUARANTINE_THRESHOLD', 25)
if score >= threshold:
    self.quarantine_manager.block_ip(ip_address, reason=f'Threat score {score} exceeded threshol
```

Why 25 points? A single honeytoken touch (+25) or a brute-force attempt combined with multi-port scanning (8 + 15 + 5 = 28) immediately quarantines the attacker before they can pivot.

3.2 Automated Quarantine & Firewall Engine (`engine/quarantine.py`)

Code & Firewall Rules:

- **Windows Implementation:** `powershell netsh advfirewall firewall add rule name="MIRAGE_BLOCK_{ip}" dir=in action=block remoteip={ip}`
- **Linux Implementation:** `bash iptables -A INPUT -s {ip} -j DROP`

CIDR Whitelist Matching Algorithm:

Uses Python's native `ipaddress` module to evaluate subnets safely:

```
import ipaddress

def is_whitelisted(self, ip_addr):
    try:
        ip_obj = ipaddress.ip_address(ip_addr)
        for entry in self.whitelist:
            if entry in ('localhost',):
                if ip_addr in ('127.0.0.1', '::1'): return True
                continue
            try:
                network = ipaddress.ip_network(entry, strict=False)
                if ip_obj in network: return True
            except ValueError:
                if ip_addr == entry: return True
        return False
    except ValueError:
        return False
```

Graceful Degradation:

If the honeypot daemon is executed in a non-elevated user shell or inside a serverless container where OS firewall commands fail, the engine catches `subprocess.CalledProcessError`, logs a warning, and **always persists the quarantine record to the SQLite database**. This guarantees system stability and demo availability regardless of host privileges.

3.3 Multi-Protocol Honeypot Traps (`honeypots/`)

1. HTTP Web Decoy (`honeypots/http_decoy.py` — Port 8080)

- Serves an authentic-looking internal corporate portal.
- Fake administrative endpoints: `/admin`, `/login`, `/wp-login.php`, `/phpmyadmin/`.
- Fake credential harvesting: Any login POST records the username/password to `credentials` table and returns a generic "Invalid credentials" error to keep the attacker probing.
- Planted `/.env` trap: Accessing `http://host:8080/.env` returns fake AWS and database secrets while secretly triggering a `honeytoken_trigger` (+25 pts).

2. SSH Trap (`honeypots/ssh_trap.py` — Port 2222)

- Built on `paramiko.ServerInterface`.
- Accepts any credentials (or default combos like `root:root`, `admin:admin`) to entice the attacker inside.
- Serves an interactive fake Linux shell prompt (`root@corp-gateway:~#`).
- Implements simulated built-in commands: `whoami`, `id`, `pwd`, `ls`, `uname -a`, `cat /etc/passwd`, `exit`.
- Emits realistic Linux outputs while silently logging every typed keystroke.
- Persists RSA host key in `config/ssh_host_key` to prevent SSH client MITM warnings.

3. FTP Trap (`honeypots/ftp_trap.py` — Port 2121)

- Built on `pyftpdlib`.
- Allows anonymous login or dictionary credentials.
- Exposes `ftp_root/credentials.xlsx` and `ftp_root/backup_db_2026.sql`.
- Overrides `on_file_sent` handler: The second an attacker downloads a decoy file via FTP, the trap fires a `honeytoken_trigger` and dispatches threat points.

3.4 Honeytoken Watchdog Monitor (`engine/honeytoken_monitor.py`)

- Utilizes Python's `watchdog.observers.Observer` to monitor filesystem access.
- Watches local directory `bait/` containing:
- `bait/admin_passwords.txt`
- `bait/backup_database.sql`
- `bait/ssh_keys/id_rsa`
- When any process reads, modifies, or opens these files, the watchdog callback immediately fires:
- Inserts audit row into `honeytoken_triggers`.
- Inserts event row into `events` table (protocol: `HONEYTOKEN`, technique: `T1083`).
- Invokes `ScoringEngine.calculate_threat_score()`.

3.5 GeoIP Intelligence Enricher (`engine/geo_enricher.py`)

- Automatically queries `http://ip-api.com/json/{ip}` upon first contact with a new IP.
- Extracts: `country`, `city`, `isp`, `latitude`, `longitude`.
- **Local/Private IP Bypass:** Automatically detects `127.0.0.1`, `localhost`, and private RFC1918 subnets (`10.0.0.0/8`, `192.168.0.0/16`, `172.16.0.0/12`) using `ipaddress.ip_address(ip).is_private` and returns `"Local Network"` without consuming external API quota.
- Caches lookups in memory for 1 hour (`GEO_CACHE_DURATION = 3600`).

3.6 MITRE ATT&CK Mapping Engine (`engine/mitre_tagger.py`)

- Reads mapping definitions from `config/mitre_mapping.json`.
- Maps observed behaviors to standardized Adversary Tactics, Techniques, and Common Knowledge (ATT&CK):
- `port_scan` ➔ **T1046** (Network Service Discovery)
- `brute_force` ➔ **T1110** (Brute Force)
- `default_creds` ➔ **T1078.001** (Default Accounts)
- `env_access` ➔ **T1552.001** (Credentials in Files)

- `cat_passwd` ➔ **T1087.001** (Local Account Discovery)
- `whoami` ➔ **T1033** (System Owner/User Discovery)
- `ftp_download` ➔ **T1005** (Data from Local System)
- `honeytoken` ➔ **T1083** (File and Directory Discovery)

3.7 Real-Time Alert Manager (`engine/alert_manager.py`)

- Formats incident summaries into structured Markdown messages.
- Dispatches HTTP POST requests to `https://api.telegram.org/bot<TOKEN>/sendMessage`.
- Delivers push notifications with audible vibration to the analyst's smartphone within 500 milliseconds of attack detection.

3.8 Concurrency-Safe Database Layer (`database/`)

- Built on SQLite 3.
- **Write-Ahead Logging (WAL):** `sql PRAGMA journal_mode = WAL; PRAGMA busy_timeout = 5000;` *Why WAL mode?* In multi-threaded honeypots, the HTTP trap, SSH trap, FTP trap, and dashboard web worker read and write simultaneously. Standard SQLite locks the entire database file during writes, causing `sqlite3.OperationalError: database is locked`. WAL mode enables **concurrent readers and writers** without locking contention.
- **Relational Tables & Schema:**
- `events`: Primary attack log stream.
- `attackers`: Aggregated profile per IP (threat score, geo coordinates, quarantine status).
- `credentials`: Captured credentials repository.
- `quarantine_log`: History of firewall blocking actions and expiration states.
- `honeytoken_triggers`: High-fidelity tripwire telemetry.
- **Performance Indexes:**
- `idx_events_ip ON events(ip_address)`
- `idx_events_timestamp ON events(timestamp)`
- `idx_quarantine_active ON quarantine_log(is_active)`
- `idx_events_protocol ON events(protocol)`

---

4. College Viva & Evaluator Defense Guide

Top 10 Questions Evaluators Will Ask & The Exact Answers to Give:

Q1: "What is unique about your honeypot compared to standard open-source honeypots like Cowrie or Dionaea?"

> **Answer:**
> *"Sir, traditional honeypots like Cowrie are specialized trap daemons that generate flat log files. They lack integrated real-time behavioral risk scoring and automatic OS firewall response. MIRAGE Junior combines a multi-protocol deception layer with an automated threat quantification pipeline that actively drops malicious connections at the OS firewall level the moment risk thresholds are breached."*

Q2: "Why did you choose a 25-point threshold for auto-quarantine?"

> **Answer:**
> *"Our scoring matrix assigns +25 points to Honeytoken triggers and +15 points to lateral cross-service scanning. A threshold of 25 points means that any attacker attempting high-confidence malicious acts (such as stealing `.env` credentials or attacking multiple trap ports simultaneously) is quarantined immediately on their first critical violation, preventing any potential lateral reconnaissance into production systems."*

Q3: "What prevents your auto-quarantine from blocking legitimate users or admins?"

> **Answer:**
> *"Two layers: First, our Honeytokens have zero legitimate use cases—no legitimate employee has any reason to access `admin_passwords.txt` or `/.env`. Second, our quarantine engine implements CIDR subnet whitelisting using Python's native `ipaddress` module. Localhost and private administrator subnets (`192.168.0.0/16` , `10.0.0.0/8`) are permanently immune."*

Q4: "How does your system handle high concurrency when multiple attackers hit simultaneously?"

> **Answer:**
> *"We enabled SQLite Write-Ahead Logging (`PRAGMA journal_mode=WAL`) and a 5000ms busy timeout. WAL allows concurrent readers and writers to operate without database lock collisions across our multi-threaded trap daemons."*

Q5: "Why did you map events to the MITRE ATT&CK framework?"

> **Answer:**
> *"MITRE ATT&CK is the global industry standard for cyber threat intelligence. Mapping our honeypot*

> *telemetry to MITRE techniques (like T1110 for brute-force and T1083 for honeytokens) allows our data to be ingested directly by enterprise SIEMs like Splunk or Microsoft Sentinel without format conversion."*

Q6: "What happens if the honeypot is run without Administrator or root privileges?"

> **Answer:**
> *"Our engine implements graceful degradation per PRD Section 15. If OS firewall commands fail due to non-elevated permissions, the system catches the exception, logs a warning, and continues recording the quarantine state and threat profiles in the database without crashing."*

Q7: "How is your Vercel deployment configured since Vercel is serverless?"

> **Answer:**
> *"Vercel runs on AWS Lambda containers where the filesystem is strictly read-only except for `/tmp`. We engineered `vercel_app.py` and `config/settings.py` to detect serverless runtime environments, initialize SQLite in `/tmp/mirage_junior.db`, and pre-seed simulated attack telemetry so the dashboard functions seamlessly online."*

Q8: "How does the visual attack timeline replay work?"

> **Answer:**
> *"In `dashboard/static/js/timeline.js`, our JavaScript renderer computes a chronological cumulative running score for the attacker's event stream. Each event is color-coded based on threat severity, visually demonstrating to analysts how the attacker escalated their intrusion attempts over time."*

Q9: "Can you unblock an IP if an administrator is accidentally blocked?"

> **Answer:**
> *"Yes. In the Quarantine Management page (`/quarantine`), administrators can click the 'Unblock' action next to any IP. This sends an asynchronous POST request to `/api/unblock/<ip>`, executing a firewall deletion rule (`netsh advfirewall delete rule`) and updating the database status in real-time."*

Q10: "What are the future enhancements for MIRAGE Junior?"

> **Answer:**
> *"Future enhancements include AI-driven dynamic prompt responses in the SSH shell using large language models, automated STIX/TAXII threat feed generation, and containerized deployment using Docker and Kubernetes."*